

Supervisor Call Monitoring — Setup, Run & Deploy Handbook

Audience: the IT person taking ownership. It assumes you can install software and copy-paste commands, but **not** that you know Python, React, or this codebase. Every step says what to type, what you should see, and what to do when it goes wrong.

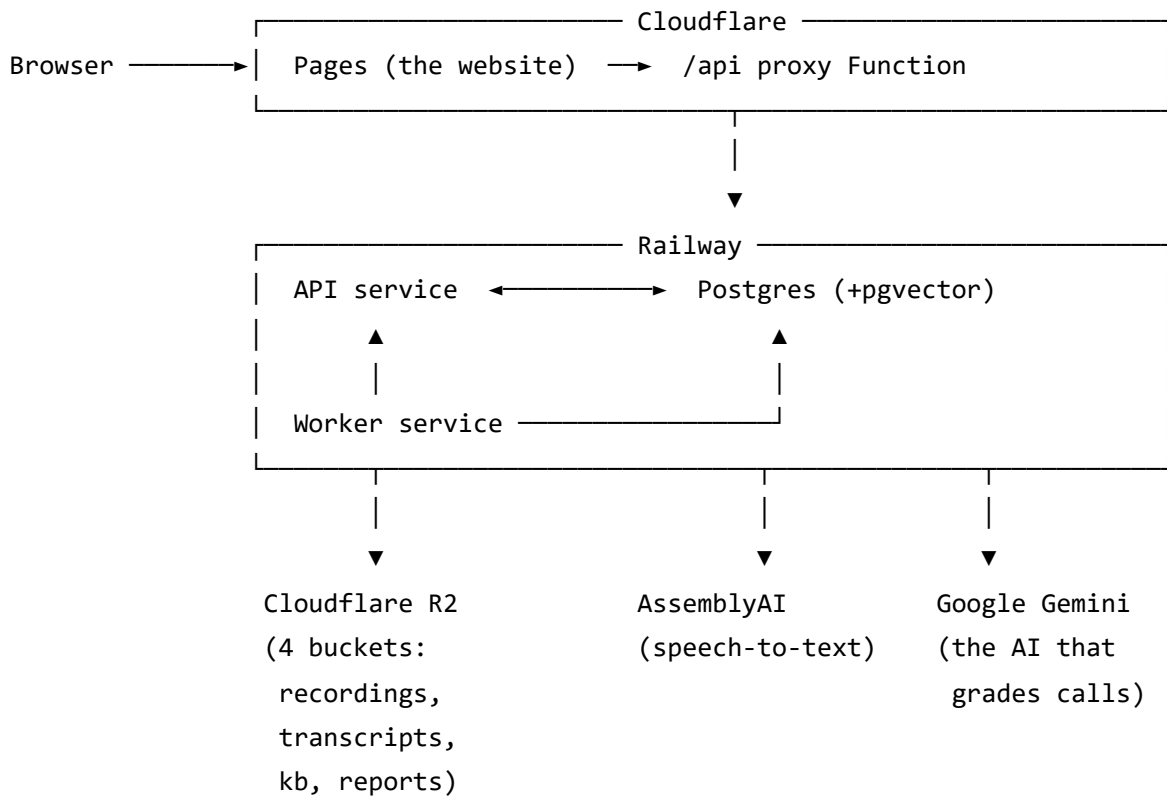
Work through it in order. Part 1 gets it running on your own machine (do this first — it is the safest place to break things). Part 2 rebuilds the whole live deployment from scratch.

Contents

Part	What it covers	Time
0. What this system is	Orientation, the moving pieces	5 min
1. Run it locally	Install, run, log in, test	45–60 min
2. Deploy from scratch	Accounts, R2, Railway, Pages	60–90 min
3. Day-2 operations	Deploys, secrets, costs, backups	—
4. Troubleshooting	Every failure we have actually hit	—

Part 0 — What this system is

It audits recorded debt-collection calls. You upload call recordings; it transcribes them, has an AI grade them against a checklist, and produces a report a supervisor can review.



Five things run:

Piece	What it does	If it stops
API	Serves the website's data, handles uploads and logins	Site stops working
Worker	Does the slow work: transcription + AI grading	Uploads queue up but never finish
Postgres	The database — all accounts, calls, reports	Everything stops
R2	File storage for audio, transcripts, reports	Uploads/downloads fail
Pages	The website itself	Nobody can reach the app

Three outside services cost money and need API keys: AssemblyAI (transcription), Google Gemini (the AI), and Cloudflare R2 (storage). Railway hosts the API/Worker/database.

0.1 The deployment that exists today

You are inheriting a running system. Before rebuilding anything, get access to these:

What	Where	Access needed
Live app	https://call-audit-supervisor.pages.dev/	An admin login, user id/email - admin@everest.local, password - EverestAdmin#2026
Source code	https://github.com/TerrorBlade2002/call-audit-supervisor (private)	Collaborator invite
API, worker, database	Railway project	Project member
Website + file storage	Cloudflare account astraglobal247 → Workers & Pages, and R2 → buckets everest-recordings / everest-transcripts / everest-kb / everest-reports	Account member
Transcription	AssemblyAI account	API key
AI	Google AI Studio	API key

GitHub access matters most: **Railway and Cloudflare Pages both deploy automatically from this repository**, so whoever can push to `main` can change production.

 **Credentials are deliberately not written in this file.** Get the current admin login from whoever is handing over, via a password manager — not email or chat. The *default* accounts created by `backend/scripts/seed_creds.py` are visible to anyone with repository access, so if production is still using them, **change the password immediately** (see 3.4).

Part 1 — Run it on your own computer

Why do this first? It proves the codebase works before you touch anything live, and it is where you should test every future change.

1.1 Install the prerequisites

Install these four, in this order. Accept the default options unless noted.

#	Software	Where	Notes
1	Git	https://git-scm.com/downloads	Used to download the code
2	Python 3.11	https://www.python.org/downloads/release/python-3119/	⚠ On Windows, tick "Add Python to PATH" on the first installer screen
3	Node.js 20 LTS	https://nodejs.org	Needed for the website
4	Docker Desktop	https://www.docker.com/products/docker-desktop/	Runs the database. Start it after installing and leave it running

Why Python 3.11 specifically? The servers run 3.11 and the automated tests run 3.11. Newer Python usually works, but if you hit a strange error, a version mismatch is a prime suspect.

Verify all four. Open a terminal (Windows: **PowerShell**; Mac: **Terminal**) and run each line. You should get a version number back, not "not found":

```
git --version
python --version
node --version
docker --version
```

Windows tip: if `python --version` opens the Microsoft Store, Python is not on your PATH. Re-run the installer, choose *Modify*, and enable "Add Python to PATH".

1.2 Get the code

If you already have the folder, skip to 1.3.

```
git clone https://github.com/TerrorBlade2002/call-audit-supervisor.git
```

```
cd call-audit-supervisor
```

This is a **private** repository — you need to be added as a collaborator first, and Git will ask you to sign in to GitHub the first time.

Everything below assumes your terminal is inside that folder.

1.3 Start the database

Make sure Docker Desktop is running (whale icon in the tray/menu bar, not spinning), then:

```
docker compose up -d db
```

You should see: Container call-qa-agent-db-1 Started .

Confirm it is healthy:

```
docker compose ps
```

You should see STATUS = Up ... (healthy) . If it says starting , wait 15 seconds and re-run.

⚠ **If this fails with a port conflict** ("port is already allocated", or the app later cannot log in with a password error), you already have PostgreSQL installed on this machine using port 5432. See [Troubleshooting → Port 5432 conflict](#).

1.4 Set up Python

Create an isolated Python environment so this project's packages don't collide with anything else:

```
python -m venv .venv
```

Activate it — **this differs by operating system**:

Windows (PowerShell):

```
.venv\Scripts\Activate.ps1
```

Mac / Linux:

```
source .venv/bin/activate
```

You should see (.venv) at the start of your terminal prompt. It must be there for every Python command below. If you close the terminal, activate again.

Windows: if you get *"running scripts is disabled on this system"*, run PowerShell as Administrator once and enter:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Now install the project:

```
pip install -e ".[dev]"
```

This takes 2–5 minutes. **You should see** Successfully installed ... at the end.

1.5 Create your settings file

The app reads its settings from a file called .env . A template is provided.

Windows (PowerShell):

```
Copy-Item .env.example .env
```

Mac / Linux:

```
cp .env.example .env
```

Open .env in any text editor (Notepad is fine). For **local testing you only need one line to be correct** — it should already match:

```
DATABASE_URL=postgresql+asyncpg://everest:everest@localhost:5432/everest
```

Leave the API keys blank for now. With no keys, the app automatically uses built-in fake transcription and fake AI, which is perfect for checking that everything runs. To test the *real* pipeline, fill in `ASSEMBLYAI_API_KEY`, `GEMINI_API_KEY`, and the four `R2_*` values from [Part 2](#).

 `.env` is deliberately excluded from Git and must **never** be committed. It holds secrets.

1.6 Create the database tables

 **This is the single most error-prone step. Read carefully.**

Because of how the very first migration was written, a **brand-new empty database** needs a different command from an **existing** one.

For a brand-new database (this is you, the first time):

```
alembic upgrade 0001
```

```
alembic stamp head
```

For a database that already has tables (e.g. you are upgrading later):

```
alembic upgrade head
```

Why? Migration `0001` builds the *entire* current set of tables in one go. Migrations `0002` onwards then try to add columns that already exist and fail with `column "... of relation ..." already exists`. `stamp head` tells the tool "we are already up to date" without re-running them. The live servers do this automatically; only local setup needs it by hand.

Verify it worked:

```
alembic current
```

You should see a revision number followed by `(head)` .

1.7 Create your login

```
python backend/scripts/seed_creds.py
```

You should see: `done` .

This creates the starter accounts. The email addresses and passwords are defined at the top of `backend/scripts/seed_creds.py` — open that file to read them.

 **These are publicly known defaults.** They are fine locally. On a live server you must change the password immediately after first login (see [3.4](#)).

1.8 Start the three processes

The app is three programs running at once. **Open three separate terminals**, and in *each one* `cd` into the project folder and activate the virtual environment (1.4) before running its command.

Terminal 1 — the API:

```
uvicorn app.main:app --reload --app-dir backend --port 8000
```

You should see: Application startup complete.

Terminal 2 — the worker (does transcription + AI grading):

```
python -m app.worker.main
```

You should see: a startup log line mentioning `worker` .

Terminal 3 — the website (does *not* need the Python environment):

```
cd frontend
```



```
npm install
```

```
npm run dev
```

You should see: Local: <http://localhost:5173/>

1.9 Check it works

1. Open <http://localhost:5173> in your browser.
2. Log in with the credentials from `backend/scripts/seed_creds.py` .
3. Create a **portfolio** (a client/process, e.g. "Key2 Recovery").
4. Open it, create a **folder** (e.g. "Trial Calls").
5. Click **Upload**, choose *Checklist only*, pick a short audio file, and upload.
6. The call should appear and move through `PENDING_TRANSCRIPTION` → `PENDING_JUDGE` → `DONE` .

Quick health check any time: open <http://localhost:8000/readyz> — it should show `{"status":"ok"}` . If it does not, the API or the database is down.

With blank API keys the transcript and report content is placeholder text. That is expected — you are testing the plumbing, not the AI.

1.10 Run the automated tests

Before you ever push a change, run what the build server runs. All four must pass:

```
ruff check backend
```

```
mypy backend/app
```

```
pytest -q
```

Then the website build, from the `frontend` folder:

```
cd frontend
```

```
npm run build
```

You should see: All checks passed! , Success: no issues found , NNN passed , and a successful build. (Run `cd ..` afterwards to get back to the project root.) If `pytest` reports tests **skipped**, the database is not reachable — see [Troubleshooting](#).

1.11 Stopping and restarting

Stop: press `ctrl+C` in each of the three terminals, then:

```
docker compose stop db
```

Restart later: `docker compose up -d db` , then repeat 1.8. You do **not** need to redo the install, migration, or seeding steps.

Part 2 — Deploy from scratch

This rebuilds the entire live system. Do it in order — later steps need values from earlier ones.

2.0 Accounts and keys you need first

Service	Purpose	Cost
GitHub	Holds the code; both hosts deploy from it	Free
Railway	Runs the API, worker, database	Paid, usage-based
Cloudflare	Website hosting (Pages) + file storage (R2)	Free tier; R2 requires a card
AssemblyAI	Speech-to-text	Pay per audio hour

Service	Purpose	Cost
Google AI Studio	Gemini API key (the AI)	Pay per token

Also generate a **JWT secret** now (this signs login sessions). Run:

```
python -c "import secrets; print(secrets.token_urlsafe(48))"
```

Copy the output somewhere safe — you will paste it as `JWT_SECRET`.

 **Keep a scratch document** as you go. You will collect ~15 values. Store it in a password manager, never in the repo.

2.1 Cloudflare R2 — file storage

- Cloudflare dashboard → **R2** → **Create bucket**. Create **four**, named exactly:
 - everest-recordings
 - everest-transcripts
 - everest-kb
 - everest-reports
- R2 → **Manage R2 API Tokens** → **Create API token**:
 - Permission: **Object Read & Write**
 - Scope it to those four buckets
 - Save three values: **Access Key ID**, **Secret Access Key**, and the **Endpoint**
(`https://<accountid>.r2.cloudflarestorage.com`)
- CORS** — the browser downloads reports and recordings straight from R2, so each bucket needs a CORS policy. Bucket → **Settings** → **CORS policy**:

```
[{ "AllowedOrigins": ["https://YOUR-PAGES-DOMAIN"],
  "AllowedMethods": ["GET", "PUT"],
  "AllowedHeaders": ["*"],
  "ExposeHeaders": ["ETag"],
  "MaxAgeSeconds": 3600 }]
```

You will not know the Pages domain until [2.6](#). **Come back and fill it in then.**

 **Record:** `R2_ENDPOINT_URL` , `R2_ACCESS_KEY_ID` , `R2_SECRET_ACCESS_KEY`

2.2 Railway — the database

⚠ **The database must have the `pgvector` extension.** This is the number-one deployment failure.

1. Railway → **New Project** → **Database** → **Add PostgreSQL**.
2. Open the database service → **Data / Query** tab → run:

```
CREATE EXTENSION IF NOT EXISTS vector;
```

3. **If that succeeds**, you are done.

If it errors with *"could not open extension control file"*, that image lacks `pgvector`.

Delete it and instead: **New** → **Empty Service** → **Deploy from Docker Image** → `pgvector/pgvector:pg16`, add a **Volume** mounted at `/var/lib/postgresql/data`, and set `POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_DB` variables.

📄 **Record:** the database service's name in Railway (you will reference it as `{{Postgres.*}}`).

2.3 Railway — the API service

1. In the same Railway project: **New** → **Deploy from GitHub repo** → select this repository.

2. **Settings:**

- **Build** → **Dockerfile Path** = `Dockerfile.api`
- **Networking** → **Generate Domain** → gives you e.g. `something.up.railway.app`.

This is your API URL.

- **Replicas = 1** (the startup command runs database migrations; more than one replica can run them simultaneously and corrupt the schema).

3. **Variables** — add each of these:

```
ENV=production
LOG_LEVEL=INFO
DATABASE_URL=postgresql+asyncpg://${Postgres.PGUSER}:${Postgres.PGPASSWORD}@${Postgres
JWT_SECRET=<the secret you generated in 2.0>
GEMINI_API_KEY=<from Google AI Studio>
GEMINI_MODEL_PRIMARY=gemini-3.6-flash
GEMINI_EMBEDDING_MODEL=gemini-embedding-001
ASSEMBLYAI_API_KEY=<from AssemblyAI>
ASSEMBLYAI_WEBHOOK_SECRET=<any long random string you invent>
PUBLIC_BASE_URL=https://<this service's generated domain>
R2_ENDPOINT_URL=<from 2.1>
R2_ACCESS_KEY_ID=<from 2.1>
R2_SECRET_ACCESS_KEY=<from 2.1>
R2_BUCKET_RECORDINGS=everest-recordings
R2_BUCKET_TRANSCRIPTS=everest-transcripts
R2_BUCKET_KB=everest-kb
R2_BUCKET_REPORTS=everest-reports
```

⚠️ Two traps in that DATABASE_URL :

- i. It must start `postgresql+asyncpg://` — **not** the plain `postgresql://` that Railway generates automatically. The app cannot connect otherwise.
- ii. `${Postgres....}` is Railway's cross-service reference syntax. Replace `Postgres` with your database service's actual name if it differs.

4. Deploy. Watch **Deploy Logs** for migrations running, then Application startup complete .
5. **Verify:** open `https://<your-api-domain>/readyz` → must return `{"status":"ok"}` .

 **Record:** the API domain.

2.4 Railway — the worker service

1. Same project: **New** → **Deploy from GitHub repo** → **the same repository again**.
(Yes — two services from one repo, different Dockerfiles.)
2. **Settings** → **Build** → **Dockerfile Path** = `Dockerfile.worker` .
Do **not** generate a domain — nothing connects to it directly.
3. **Variables:** copy **every variable** from the API service, including `PUBLIC_BASE_URL` and `ASSEMBLYAI_WEBHOOK_SECRET` . They must match exactly.

4. Deploy. The logs should show the worker starting.

The worker deliberately does **not** run migrations — only the API does, to avoid two services changing the schema at once.

2.5 Create the first login

Once the API has deployed successfully:

Railway → API service → ... **menu** → **Run a command**:

```
PYTHONPATH=backend python backend/scripts/seed_creds.py
```

This creates the admin account defined in `backend/scripts/seed_creds.py`.

 **Immediately after your first successful login, change that password** in the app's Users panel. The defaults are visible to anyone with repository access.

2.6 Cloudflare Pages — the website

1. Cloudflare → **Workers & Pages** → **Create** → **Pages** → **Connect to Git** → this repo.

2. **Build settings**:

- **Root directory**: `frontend`
- **Build command**: `npm run build`
- **Output directory**: `dist`
- Framework preset: *Vite* (or None)

3. **Settings** → **Environment variables** → **Production**:

```
API_BASE_URL = https://<your Railway API domain>
```

This is read by `frontend/functions/api/[[path]].js`, which forwards `/api/*` requests to Railway. Without it the site loads but nothing works.

4. Deploy. You get a domain like `your-project.pages.dev` — **this is the app URL** you give users.

5. **Go back to 2.1 step 3** and set the R2 CORS

`AllowedOrigins` to this domain.

2.7 Apply the storage retention rule (one-off)

Recordings and transcripts are auto-deleted after 30 days; the knowledge base and reports are kept. Run once, from your machine, with the R2 values filled into your local `.env` :

```
python scripts/setup_r2.py
```

Safe to re-run.

2.8 Final smoke test

1. Open the Pages URL and log in.
2. Create a portfolio → folder.
3. Upload one short recording, choose *Checklist only*.
4. Watch it reach `DONE` (the **Lifecycle** panel shows progress).
5. Open the report; download the HTML/PDF version.
6. Check Railway **worker logs** show the job being processed, and **API logs** show the AssemblyAI webhook arriving.

If all six pass, the deployment is complete. 

Part 3 — Day-2 operations

3.1 How deployments happen

Both hosts watch the GitHub repository. **Pushing to the `main` branch automatically redeploys both:**

```
git push origin main ──┐
                        │→ Railway rebuilds API + Worker (and runs DB migrations)
                        └─┐
                          │→ Cloudflare Pages rebuilds the website
```

There is also a test pipeline (`.github/workflows/ci.yml`) that runs the linter, type checker, tests, and an AI-quality gate on every push. **Check it is green before assuming a deploy is good.**

Recommended workflow for any change:

1. Make the change locally.
2. Run all four checks from [1.10](#).
3. Commit and push.
4. Watch the GitHub Actions run go green.
5. Verify on the live site.

3.2 Database migrations

If a change adds a database column, it ships as a file in `backend/migrations/versions/`. The API service runs `alembic upgrade head` automatically on deploy — **you normally do nothing**. Just confirm in the deploy logs that the migration ran.

3.3 Backups

Railway's Postgres has snapshot/backup settings in the database service — **turn them on**. The database is the only irreplaceable component; R2 files can be re-uploaded, but reports, accounts, and audit history cannot.

3.4 Rotating credentials

- **App admin password:** change in the app's Users panel.
- **API keys / JWT secret:** update the variable in **both** the Railway API and Worker services, then redeploy both. Changing `JWT_SECRET` logs everyone out — that is expected and is exactly what you want if a secret leaked.
- **Never** put secrets in the repository. `.env` is git-ignored for this reason.

3.5 Controlling cost

Every rate limit and spending cap is an environment variable — no code change needed. The most useful ones:

Variable	Effect
DAILY_CAP_PER_PORTFOLIO	Hard limit on calls processed per client per day. Over-cap work is postponed, not lost
GEMINI_MAX_CONCURRENCY	How many AI calls run at once — raise for speed, lower for cost
GEMINI_RPM / GEMINI_TPM / GEMINI_RPD	Guardrails against runaway AI spend
AAI_MAX_INFLIGHT	Concurrent transcriptions

Full explanations: docs/RATE_LIMITS_AND_COST.md .

Part 4 — Troubleshooting

Port 5432 is already in use

Symptom: `docker compose up` fails to bind the port, or the app reports password authentication failed for user "everest" .

Cause: PostgreSQL is already installed on this machine and owns port 5432, so you are connecting to the *wrong* database.

Fix: publish the project's database on a free port instead. Create a file named `docker-compose.override.yml` in the project root:

```
services:
  db:
    ports:
      - "5439:5432"
```

Then update the port in your `.env` :

```
DATABASE_URL=postgresql+asyncpg://everest:everest@localhost:5439/everest
```

Then `docker compose up -d db` . The override file is git-ignored — it is yours alone.

column "... " already exists when creating tables

You ran `alembic upgrade head` on an **empty** database. See [1.6](#)

— use `alembic upgrade 0001` then `alembic stamp head` instead.

If it already half-failed, reset and redo:

```
alembic stamp base
```

```
alembic upgrade 0001
```

```
alembic stamp head
```

Tests are skipped instead of passing

Symptom: `pytest` says e.g. 96 passed, 51 skipped .

Cause: the database is unreachable, so database-dependent tests skip themselves rather than fail. See the exact reason with:

```
pytest -q -rs
```

Fix: make sure Docker Desktop is running and `docker compose up -d db` succeeded, and that the port in `.env` matches. A *fully* passing run has **zero** skips.

Tests fail with `column ... does not exist` after pulling new code

The test database is stale. Drop it — it is rebuilt automatically on the next test run:

```
docker compose exec db psql -U everest -d everest -c "DROP DATABASE IF EXISTS everest_pytest;"
```

ModuleNotFoundError: No module named 'app' , or alembic prints nothing

Symptom: `python backend/scripts/seed_creds.py` or `python scripts/setup_r2.py` fails with No module named 'app' , or the alembic command silently does nothing.

Cause: the Python environment's link to this project is stale — usually because the project folder was copied, renamed, or moved after `pip install -e` was run. The link still points at the old location.

Check it:

```
pip show everest-auditor
```

The Location / Editable project location shown must be **your current project folder**. If it is not, re-link it:

```
pip install -e ".[dev]"
```

Workaround if you cannot re-install right now — tell Python where to look, then run the command:

Windows (PowerShell):

```
$env:PYTHONPATH="backend"; python backend/scripts/seed_creds.py
```

Mac / Linux:

```
PYTHONPATH=backend python backend/scripts/seed_creds.py
```

You can also always substitute `python -m alembic` for a non-working alembic .

"Upload failed (413)"

Cause: the request was too large. Cloudflare rejects any single request over **100 MB** on the Free/Pro plans, before it reaches the server.

The app now uploads **one file per request**, so this only happens if a *single* recording exceeds ~95 MB. Convert that recording to MP3 (roughly a tenth the size of WAV) or split it.

If you routinely handle recordings larger than 100 MB, the proper fix is to switch uploads to the direct-to-R2 path already present in the codebase (`uploads:presign`), which bypasses Cloudflare entirely. It needs R2 CORS configured and a small frontend change.

Migrations fail on deploy with an extension error

The Railway Postgres image lacks `pgvector`. See [2.2](#).

The site loads but everything shows an error

Almost always `API_BASE_URL` is missing or wrong on Cloudflare Pages ([2.6](#)), or the Railway API is down. Check `https://<api-domain>/readyz` first.

Uploads succeed but calls never finish processing

The **worker** is not running or is crashing. Check the Railway worker service logs. Also confirm it has the same variables as the API.

Docker Desktop is stuck / commands hang

Docker Desktop occasionally wedges. Quit it completely, reopen, and wait for the whale icon to stop animating. If that fails, use its **Troubleshoot** → **Restart** option.

Where to look next

File	What it explains
README.md	Architecture and how the AI pipeline is designed
docs/DEPLOY.md	The condensed deployment reference

File	What it explains
docs/RATE_LIMITS_AND_COST.md	Every rate limit and cost lever, with reasoning
.env.example	Every setting the app understands
.github/workflows/ci.yml	Exactly what the automated tests run